HelixCode — Open Issues Tracker

> Per Constitution §11.4.15 (Item-status tracking) + §11.4.16 (Item-type tracking) + §11.4.19 (Fixed-document column-alignment) + CONST-057 (Type-aware closure vocabulary) + CONST-058 (Reopened-source attribution).
>
> **Authoritative resumption ledger**: `docs/CONTINUATION.md` (CONST-044). This file complements it with item-level granularity for currently-open work.
>
> **Status vocabulary** (closed set): `Queued` | `In progress` | `Ready for testing` | `In testing` | `Reopened` | `Fixed/Implemented/Completed` `(→ Fixed.md)`
>
> **Type vocabulary** (closed set): `Bug` | `Feature` | `Task`

Prefix convention

Round 189 (2026-05-19) introduced per-project / per-submodule ID prefixes replacing the legacy `ISSUE-NNN` flat namespace. New items MUST use the prefix matching their scope; cross-cutting items affecting two or more submodules (or any meta-repo concern) live under `HXC`. Numeric portion is per-prefix (each prefix starts at `001`). Legacy `ISSUE-NNN` IDs renamed forward-only per CONST-043; historical close-out narratives in `docs/CONTINUATION.md` preserve original IDs verbatim, and `docs/Fixed.md` annotates each closure with `(ex-ISSUE-NNN)` for git-history traceability.

Prefix	Scope	Source
HXC	HelixCode root project (project-wide, multi-submodule,	this repo

Prefix	Scope	Source
	governance, infrastructure)	
HXA	HelixAgent submodule	`submodules/helix_agent` (when present) / `helix_agent/` tree
HXM	HelixMemory submodule	`submodules/helix_memory`
HXL	HelixLLM submodule	`submodules/helix_llm`
HXQ	HelixQA submodule	`submodules/helix_qa` (`helix_qa/`)
HXS	HelixSpecifier submodule	`submodules/helix_specifier`
HXO	HelixOrchestrator (= LLMOrchestrator) submodule	`submodules/llm_orchestrator`
HXV	HelixVerifier (= LLMsVerifier) submodule	`submodules/llms_verifier`
HXD	HelixDocProcessor (= DocProcessor) submodule	`submodules/doc_processor`
HXI	HelixI18n (when added)	tba
PLN	Planning submodule (vasic-digital)	`submodules/planning`
VEN	VisionEngine submodule (HelixDevelopment)	`submodules/vision_engine`
SLF	SelfImprove submodule (vasic-digital)	`submodules/self_improve`
STO	Storage submodule (vasic-digital)	`submodules/storage`
OPS	LLMOps submodule (vasic-digital)	`submodules/llm_ops`

Prefix	Scope	Source
VDB	VectorDB submodule (vasic-digital)	`submodules/vector_db`
OBS	Observability submodule (vasic-digital)	`submodules/observability`
MCP	MCP_Module submodule (vasic-digital)	`submodules/mcp_module`
MSG	Messaging submodule (vasic-digital)	`submodules/messaging`
MDW	Middleware submodule (vasic-digital)	`submodules/middleware`
PLG	Plugins submodule (vasic-digital)	`submodules/plugins`
STR	Streaming submodule (vasic-digital)	`submodules/streaming`
WAT	Watcher submodule (vasic-digital)	`submodules/watcher`
CNV	conversation submodule (vasic-digital)	`submodules/conversation`
AUT	Auth submodule (vasic-digital)	`submodules/auth`
LZY	Lazy submodule (vasic-digital)	`submodules/lazy`
ATP	AutoTemp submodule (vasic-digital)	`submodules/auto_temp`
PLI	PliniusCommon submodule (vasic-digital)	`submodules/plinius_common`
CHL	challenges submodule (vasic-digital)	`challenges/`
CNT		`containers/`

Prefix	Scope	Source
	containers submodule	
SEC	security submodule	`security/`
PAN	panoptic submodule	`panoptic/`

For submodules not listed above, default to the first 3 letters of the submodule name, uppercase (e.g. `Watcher` → `WAT`). Document the new prefix in this table on first use.

Legacy → new mapping (round 189)

Old ID	New ID	Scope rationale
ISSUE-001	VEN-001	VisionEngine `helix-gitlab` URL
ISSUE-002	VEN-002	VisionEngine `vasic-digital-github` fork divergent
ISSUE-003	HXL-001	HelixLLM `analysis_test.go` hardcoded path
ISSUE-004	HXL-002	HelixLLM TOON `WriteTOON` 500
ISSUE-005	HXC-001	CONST-052 rename programme (project-wide)
ISSUE-006	HXC-002	Round-74 residual LOGIC FAILs (multi-submodule cross-cutting)
ISSUE-007	HXC-003	CONST-046 migration backlog (project-wide)
ISSUE-008	HXQ-001	helix_qa `TestPerformance` flake
ISSUE-009	HXA-001	helix_agent handler tests (4)
ISSUE-010	HXA-002	helix_agent debate API drift
ISSUE-011	HXA-003	venice CONST-037 (in helix_agent)

HXC-119 — Agent Client Protocol (ACP) support is absent from the platform

Status: Queued **Type:** Feature **Severity:** High **Created-By:** Claude

Governance rule CONST-040 lists the Agent Client Protocol among required capabilities, but there is no implementation of it anywhere in the codebase. Any user or integration expecting ACP connectivity currently cannot use it. The work is to design and implement real ACP support, or, if it proves structurally infeasible, to document that with cited evidence. The platform will then either genuinely support ACP or hold an honest, evidenced position instead of an unmet claim.

HXC-122 — Memory and automation test suites mostly skip themselves without a running server

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

Two categories of tests, memory-usage and end-to-end automation, skip most of their cases by default because they require a live server or special environment flags not set in normal runs. In practice these areas are largely unverified even though the tests appear to exist. The work provides a documented, repeatable way to run them against real infrastructure so they actually execute and prove the behavior. Memory and automation behavior then becomes genuinely tested rather than merely scaffolded.

HXC-136 — Verify the remaining automated test types run with real captured evidence

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

Several mandated automated test categories — load/denial-of-service, scaling, stress and chaos, and user-interface/experience — were not exercised in the latest real-infrastructure run, so their current health is unconfirmed. The work is to run each of these test types against real infrastructure and capture proof of the results. This completes the promised full test-type coverage and confirms the product holds up under load and adverse conditions.

HXC-138 — Run the end-to-end challenge suite against a running server

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

The end-to-end challenge runner can now launch all its scenarios (a missing option was just fixed), but the scenarios still need to be executed against a live server with a real model to confirm the complete user journeys work. The work is to stand up a server and run the challenges, capturing the results. This provides real proof that the headline user workflows function end to end.

HXC-144 — Server leaks goroutines under sustained DDoS-flood load (chaos test)

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

Under the sustained request-flood chaos test against the real running server, the goroutine count grew by 5 which exceeds the tolerance of 4, signalling a goroutine leak in a request-handling path when the server is hammered. Left unaddressed this degrades long-running server stability under load. The work is to find the leaking goroutine (likely an unclosed channel, context, or connection in a hot handler) and fix it so the count stays within tolerance under flood. Evidence: docs/qa/infra_retest_20260712_hxc122_138/EVIDENCE.md (Server 7/8).

HXC-145 — Configured Xiaomi model mimo-v2-flash is rejected by the real Xiaomi API

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

During the real-infra retest the Xiaomi provider chaos tests failed 2 of 5 because the model id configured for Xiaomi (mimo-v2-flash) is rejected by the live Xiaomi API, indicating the configured model name is stale or wrong. Users selecting the Xiaomi provider with that model would get errors. The work is to determine the correct current Xiaomi model id (from the provider or the verifier as single source of truth) and update the configuration so Xiaomi requests succeed. Evidence: docs/qa/infra_retest_20260712_hxc122_138/EVIDENCE.md (Xiaomi 3/5).

HXC-146 — E2E challenge runner interfaces (cli/rest/tui/websocket) do not drive the real server HTTP API

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

The e2e challenge runner advertises multiple interface modes (cli, rest, tui, websocket) but none of them actually exercises the HelixCode server's real HTTP API during a run, so the challenges validate the runner's own logic rather than the shipped server endpoints. This is a documentation-versus-implementation gap that weakens the end-to-end proof. The work is to wire the challenge runner's interfaces to genuinely call the running server's HTTP API so the challenges prove the real user-facing endpoints work. Discovered 2026-07-12 real-infra retest. Evidence: docs/qa/infra_retest_20260712_hxc122_138/hxc138_challenge_report.json.

HXC-147 — OpenRouter provider automation test nil-pointer panics on stale model deepseek-r1-free

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

Running the (now-compilable) automation test binary against the live OpenRouter API, TestAllFreeProvidersAutomation Provider_OpenRouter BasicGeneration panics with a nil-pointer dereference: the configured free model id deepseek-r1-free is stale/rejected and the code path is missing a nil-check on the error before using the response. Users of the OpenRouter free provider with that model would hit the same crash. The work is to correct the free-provider model id (sourced from the verifier as single source of truth) and add the missing nil-check so a rejected model degrades gracefully instead of panicking. NOTE this environment has live provider API keys set so provider tests spend real money; guard/skip accordingly. Found 2026-07-12.

HXC-148 — Wire RAG retrieval-augmentation into the OpenAI/Anthropic wire-facade endpoints

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

HXC-118 wired Retrieval-Augmented-Generation into the native server generate and stream endpoints and the CLI, but the OpenAI-compatible and Anthropic-compatible wire-facade endpoints (/v1/chat/completions and /v1/messages) still bypass RAG entirely, so clients using those compatibility surfaces do not get retrieval-augmentation even when it is enabled. The work is to apply the same applyRAGContext wiring to those facade handlers so RAG behaves consistently across every generate surface. This is a smaller secondary surface than the native path already fixed. Found during HXC-118 review 2026-07-12.